

Floyd Cycle Detection Algorithm + LC287 Complete Notes

1. What is Floyd Cycle Detection?

Also called Tortoise and Hare Algorithm.

Uses two pointers:

Slow pointer:
moves one step

Fast pointer:
moves two steps

If a cycle exists, both pointers meet.

Time: $O(n)$
Space: $O(1)$

2. Linked List Cycle Idea

Originally used for linked lists.

`slow = slow.next`

`fast = fast.next.next`

If they meet:
cycle exists.

3. Why LC287 Uses Floyd?

Leetcode 287:
Find Duplicate Number.

Array:

`[1,3,4,2,2]`

Think:

index -> nums[index]

Duplicate value creates multiple incoming edges,
which forms a cycle.

4. Algorithm Steps

Phase 1:
Find collision.

slow = nums[slow]

fast = nums[nums[fast]]

Phase 2:
Find cycle entrance.

Move slow to beginning.

Move both one step.

Meeting point is duplicate.

5. Mathematical Proof

Let:

a = distance before cycle

b = distance inside cycle before meeting

c = remaining cycle length

Slow travels:

a+b

Fast travels twice.

This proves:

one pointer from start

+

one pointer from meeting point

meet at cycle entrance.

6. LC287 Approaches

Sorting:
 $O(n \log n)$

HashSet:
 $O(n)$ space

Binary Search:
 $O(n \log n)$

Floyd:
 $O(n)$ time
 $O(1)$ space

Best approach.

7. Pattern Recognition

Use Floyd when:

- Cycle detection
- Linked list loop
- Array values act like pointers
- Need constant space

8. Related Problems

LC141 Linked List Cycle

LC142 Linked List Cycle II

LC287 Find Duplicate Number

LC202 Happy Number

LC457 Circular Array Loop

Python LC287 Code

```
def findDuplicate(nums):
```

```

slow=nums[0]
fast=nums[0]

while True:

    slow=nums[slow]

    fast=nums[nums[fast]]

    if slow==fast:
        break

slow=nums[0]

while slow!=fast:

    slow=nums[slow]

    fast=nums[fast]

return slow

```

C++ LC287 Code

```

int findDuplicate(vector<int>& nums){
int slow=nums[0];
int fast=nums[0];

do{
slow=nums[slow];
fast=nums[nums[fast]];

}while(slow!=fast);

slow=nums[0];

while(slow!=fast){
slow=nums[slow];
fast=nums[fast];
}

return slow;
}

```

Java LC287 Code

```

class Solution{
public int findDuplicate(int[] nums){
int slow=nums[0];
int fast=nums[0];

```

```
do{
    slow=nums[slow];
    fast=nums[nums[fast]];

}while(slow!=fast);

slow=nums[0];

while(slow!=fast){
    slow=nums[slow];
    fast=nums[fast];
}

return slow;
}
}
```

Two Pointer Family

Floyd is part of two pointer techniques:

1. Fast Slow pointer
2. Left Right pointer
3. Sliding Window
4. Merge pointers